

LAB

Implement a text embedding-based search system using Google Colab

Contents

Introduction	2
Software requirements	3
Objective	3
Lab steps	3
Estimated duration to complete	3
Scenario	4
Background information	4
Challenge.....	4
Solution	4
Step 1: Train the Word2Vec model to generate word embeddings	5
Overview.....	5
Instructions	5
Step 2: Visualize word embeddings using dimensionality reduction techniques	13
Overview.....	13
Instructions	13
Conclusion.....	22

Introduction

In this lab, you'll build a text embedding-based search and visualization system using Google Colab. You'll use Word2Vec, a popular text embedding model, to explore how it captures semantic relationships between words by embedding them and how changing model parameters affects these relationships.

Next, you'll apply two dimensionality reduction techniques, principal component analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE), to visualize word similarity.

You'll first use PCA, and then replace it with t-SNE for better-defined clustering. This iterative approach will help demonstrate the impact of each technique on clustering and semantic separation.

Software requirements

To complete this lab, you don't have to install any software on your computer. You only require a Google account to access Google Colab. If you don't have a Google account, you must create one.

Objective

After completing this lab, you should be able to:

- Implement a text embedding-based search system to retrieve relevant results

Lab steps

This lab requires you to complete the following procedures:

- Train the Word2Vec model to generate word embeddings
- Visualize word embeddings using dimensionality reduction techniques

Estimated duration to complete

30 minutes

Scenario

Background information

You're a data analyst at AutoTrend Analytics, a growing automobile market research firm. The company uses data to help major car manufacturers understand consumer preferences and industry trends, enabling them to make more informed business decisions.

As part of the data analytics team, your responsibilities include exploring which car features matter most to the two major segments of car buyers: those who prefer budget-friendly cars and those who prefer luxury ones.

Challenge

Currently, your team relies on manual analysis and simple keyword searches to find these insights, which presents two major problems:

- **Limited semantic insight:** Without advanced models, the analysis fails to capture nuanced word relationships. For example, terms such as economical, basic, and practical might be treated as unrelated, even though they commonly describe budget-friendly vehicles.
- **Poor scalability and visualization:** As text volume increases, the current process becomes time consuming and difficult to interpret. The marketing team is struggling to analyze spreadsheets and static charts, as they don't provide clear visualizations that might help the team draw meaningful conclusions.

This approach limits your team's ability to provide actionable recommendations that can drive product improvements, enhance customer satisfaction, and improve sales.

Solution

To address these issues, you propose a new AI-driven approach, which includes the following steps:

- Training Word2Vec on text data that contains the words customers use to describe budget-friendly and luxury cars
- Tweaking parameters to observe how word relationships shift in context
- Applying dimensionality reduction techniques such as PCA and t-SNE to reduce dimensions and visualize clusters of related words

This approach enables deeper insights into the language consumers use to describe their preferences, helping your team identify the emotional and practical associations they make with various categories of cars, such as budget or luxury. Your work will help AutoTrend deliver more strategic, data-driven advice to its clients.

You'll implement and test these solutions in this lab.

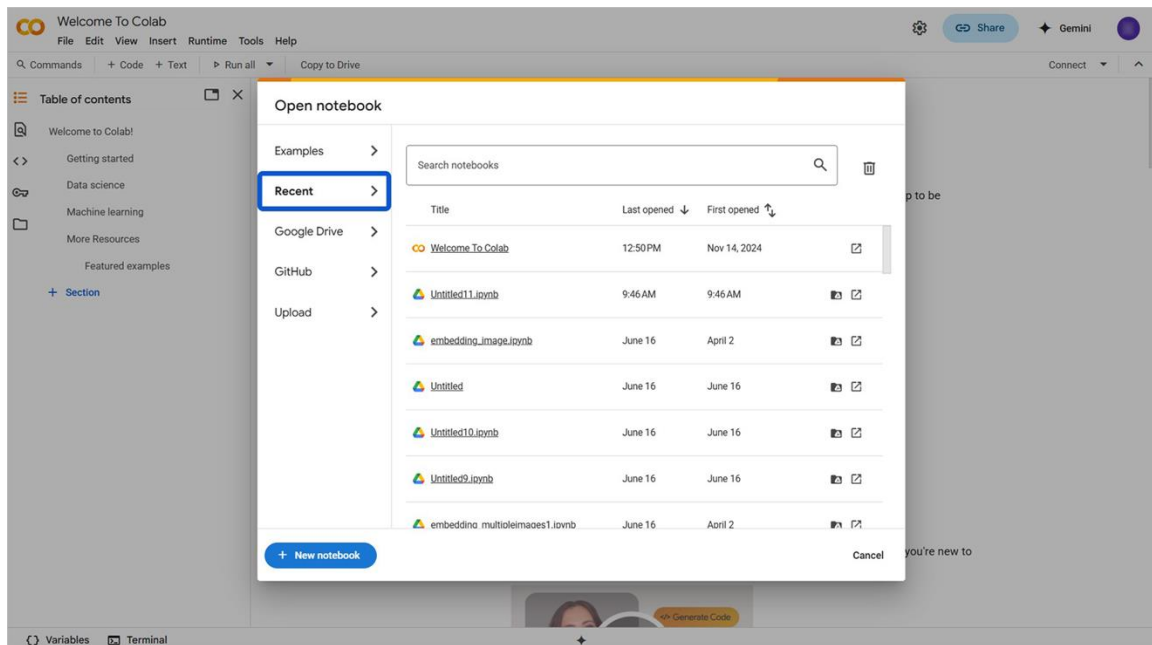
Train the Word2Vec model to generate word embeddings

Overview

In this procedure, you'll uncover patterns from the words customers use by training the Word2Vec model on a sample corpus using default parameters to generate word embeddings. To do so, you'll install the gensim and matplotlib libraries, import the Word2Vec model, define the corpus, and train the Word2Vec model using the default parameters.

Instructions

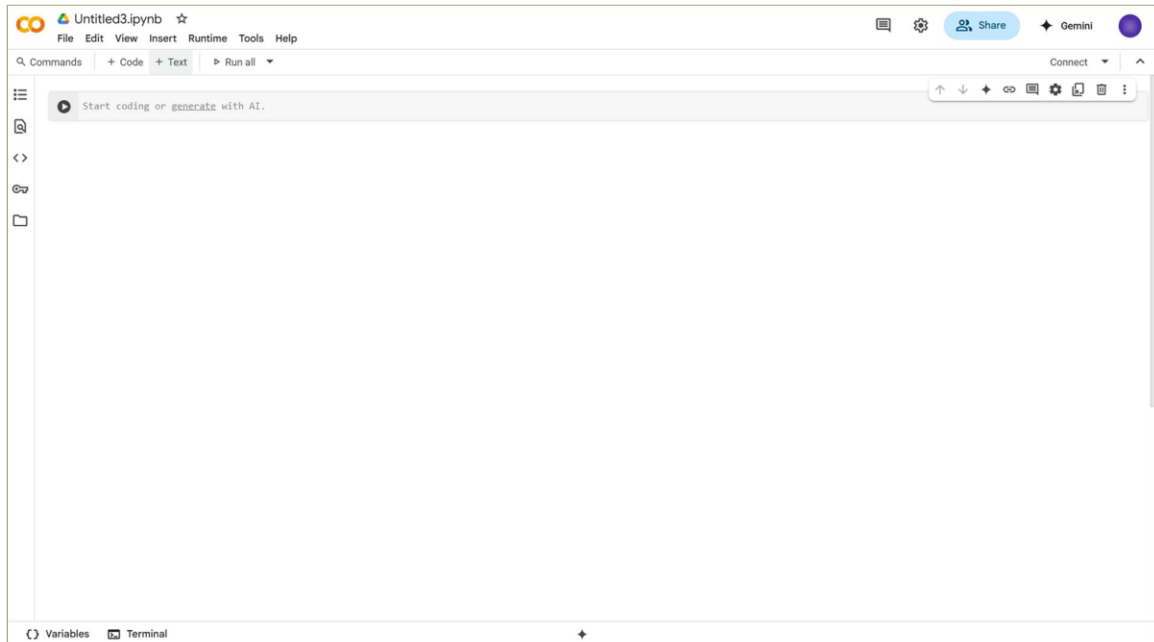
1. Go to the [Google Colab](https://colab.research.google.com/) website. The Open notebook window is displayed.



Select **New notebook**.

Result: A new notebook with the first code cell is displayed, as shown in the following screenshot.

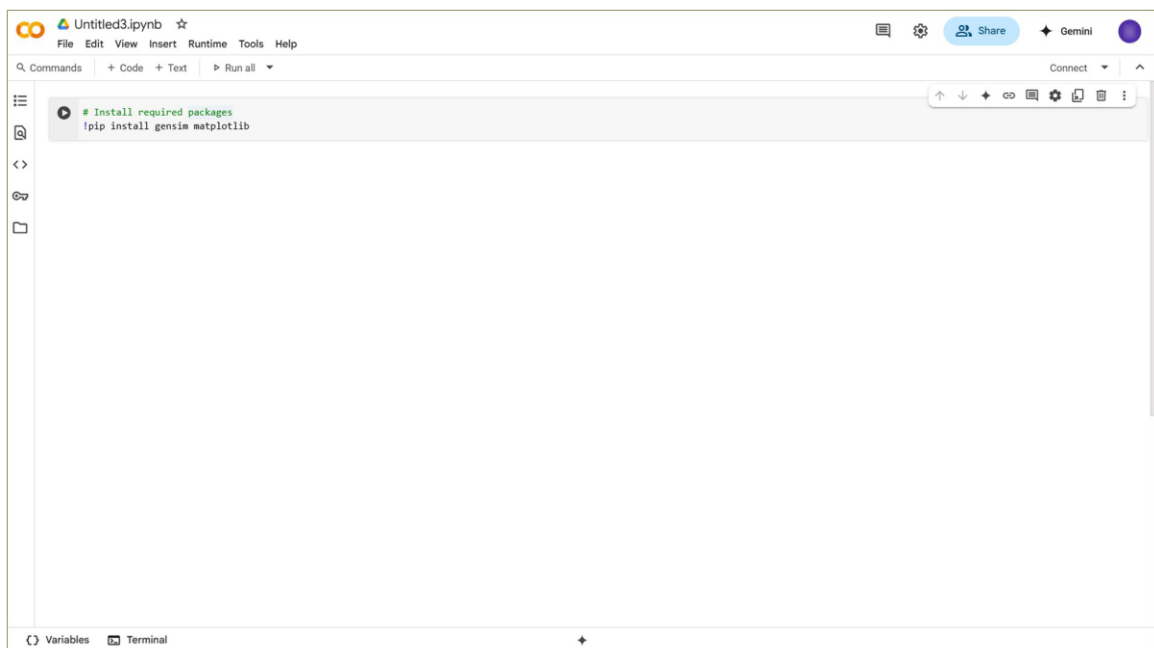
Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**



2. Install gensim and matplotlib, as these Python libraries might not be available in Google Colab by default. To install them, copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

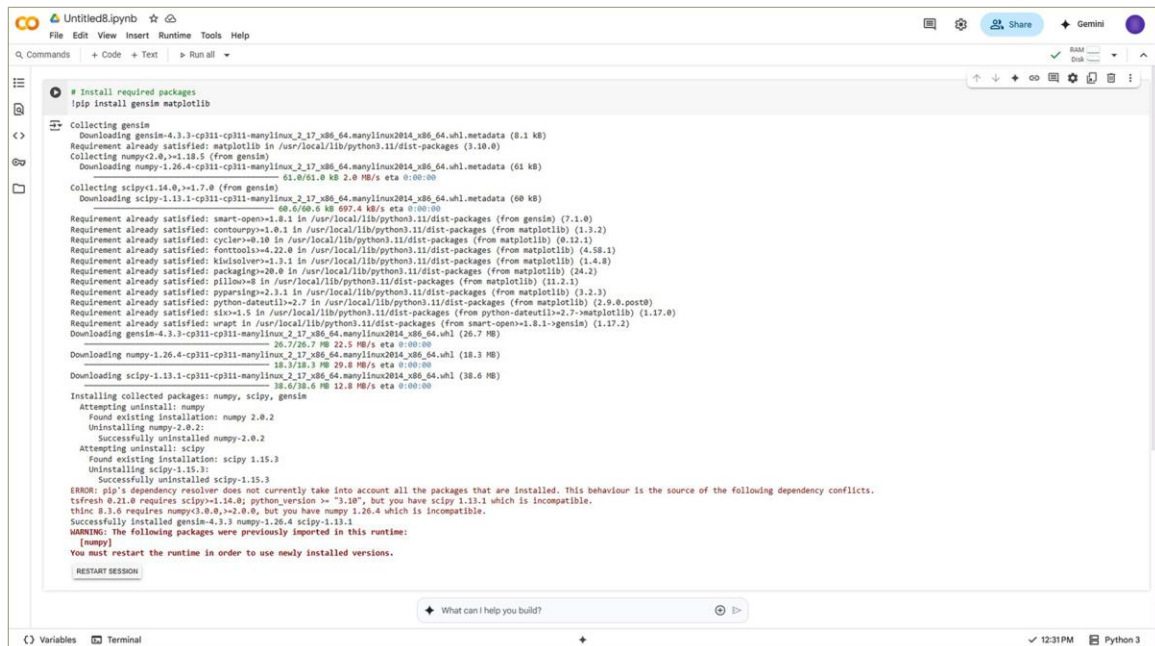
```
# Install required packages  
!pip install gensim matplotlib
```

The following screenshot shows the code pasted into the code cell.



Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**

Result: The model generates the output that confirms you successfully installed gensim and matplotlib, as shown in the following screenshot.



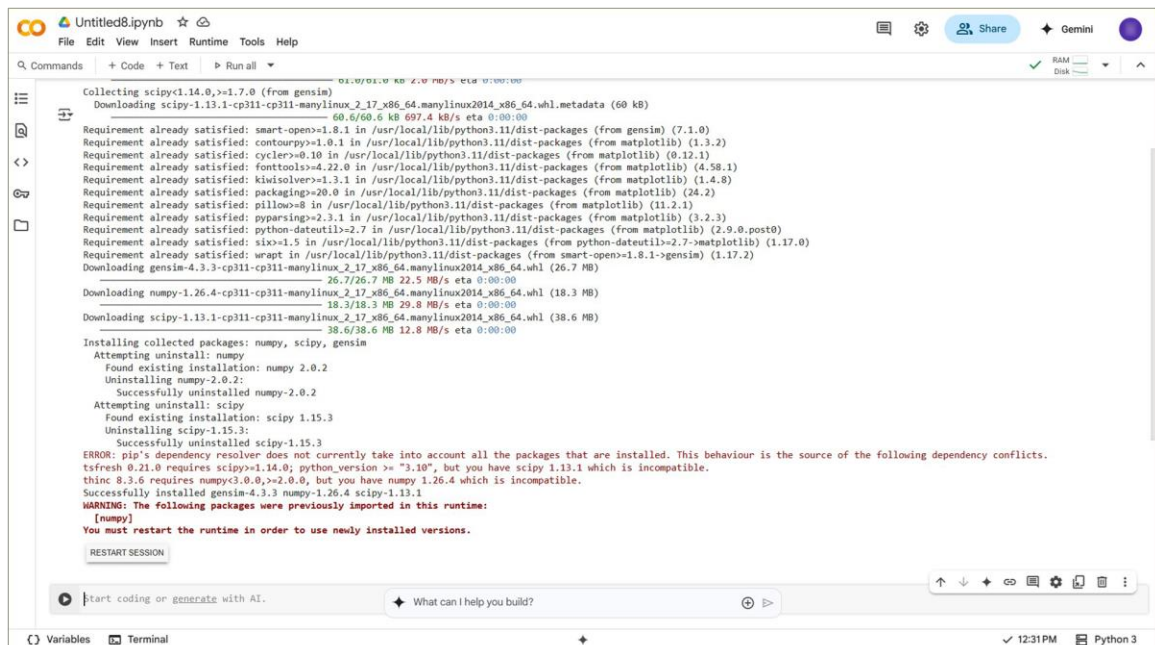
```
# Install required packages
!pip install gensim matplotlib

Collecting gensim
  Downloading gensim-4.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.1 kB)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
Collecting numpy<2.0.0, >=1.18.5 (from gensim)
  Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
Collecting scipy<1.14.0, >=1.7.0 (from gensim)
  Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.11/dist-packages (from gensim) (7.1.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.58.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packages (from smart-open>=1.8.1->gensim) (1.17.2)
Downloading gensim-4.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (26.7 MB)
  26.7/26.7 MB 22.5 MB/s eta 0:00:00
Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.3 MB)
  18.3/18.3 MB 29.8 MB/s eta 0:00:00
Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.6 MB)
  38.6/38.6 MB 12.8 MB/s eta 0:00:00
Installing collected packages: numpy, scipy, gensim
Attempting uninstall: numpy
  Found existing installation: numpy 2.0.2
  Uninstalling numpy-2.0.2:
    Successfully uninstalled numpy-2.0.2
Attempting uninstall: scipy
  Found existing installation: scipy 1.15.3
  Uninstalling scipy-1.15.3:
    Successfully uninstalled scipy-1.15.3
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
  tsfresh 0.21.0 requires scipy>=1.14.0; python_version >= "3.10", but you have scipy 1.13.1 which is incompatible.
  thinc 8.3.6 requires numpy<3.0.0, >=2.0.0, but you have numpy 1.26.4 which is incompatible.
Successfully installed gensim-4.3.3 numpy-1.26.4 scipy-1.13.1
WARNING: The following packages were previously imported in this runtime:
[numpy]
You must restart the runtime in order to use newly installed versions.

RESTART SESSION
```

3. Import the Word2Vec model. To do so, add a new cell in the notebook. Select the **+ Code** tab.

The following screenshot shows a new cell in the notebook after the previous output.



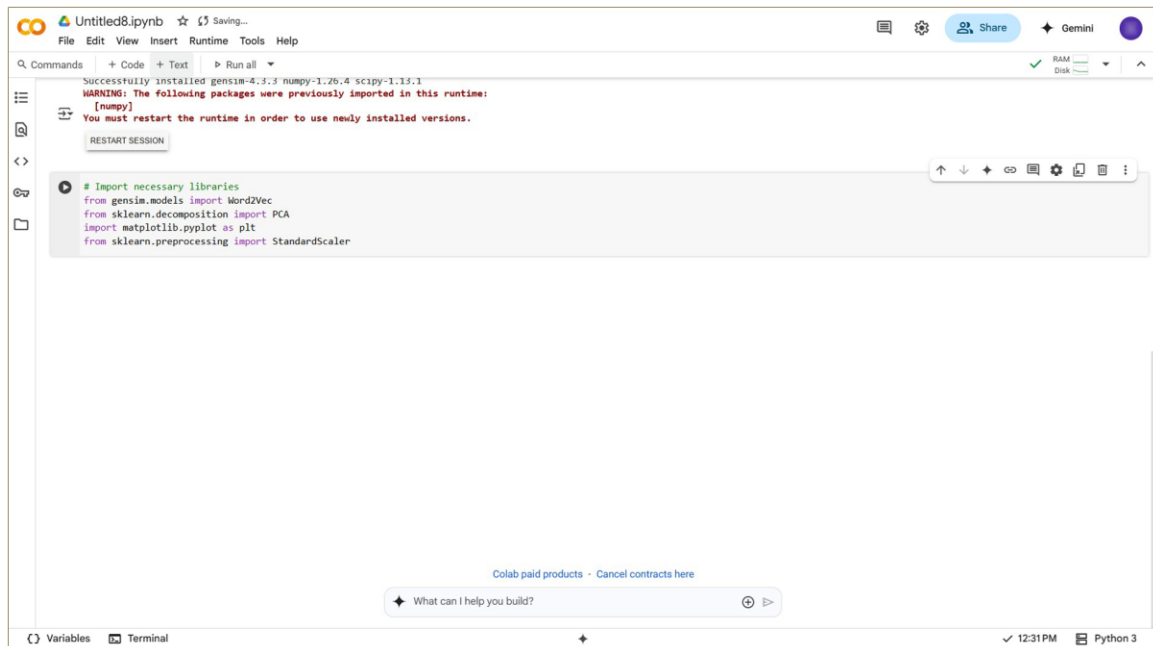
```
Collecting scipy<1.14.0, >=1.7.0 (from gensim)
  Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
Requirement already satisfied: smart-open>=1.8.1 in /usr/local/lib/python3.11/dist-packages (from gensim) (7.1.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.58.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.11/dist-packages (from smart-open>=1.8.1->gensim) (1.17.2)
Downloading gensim-4.3.3-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (26.7 MB)
  26.7/26.7 MB 22.5 MB/s eta 0:00:00
Downloading numpy-1.26.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.3 MB)
  18.3/18.3 MB 29.8 MB/s eta 0:00:00
Downloading scipy-1.13.1-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.6 MB)
  38.6/38.6 MB 12.8 MB/s eta 0:00:00
Installing collected packages: numpy, scipy, gensim
Attempting uninstall: numpy
  Found existing installation: numpy 2.0.2
  Uninstalling numpy-2.0.2:
    Successfully uninstalled numpy-2.0.2
Attempting uninstall: scipy
  Found existing installation: scipy 1.15.3
  Uninstalling scipy-1.15.3:
    Successfully uninstalled scipy-1.15.3
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
  tsfresh 0.21.0 requires scipy>=1.14.0; python_version >= "3.10", but you have scipy 1.13.1 which is incompatible.
  thinc 8.3.6 requires numpy<3.0.0, >=2.0.0, but you have numpy 1.26.4 which is incompatible.
Successfully installed gensim-4.3.3 numpy-1.26.4 scipy-1.13.1
WARNING: The following packages were previously imported in this runtime:
[numpy]
You must restart the runtime in order to use newly installed versions.

RESTART SESSION
```

4. Copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

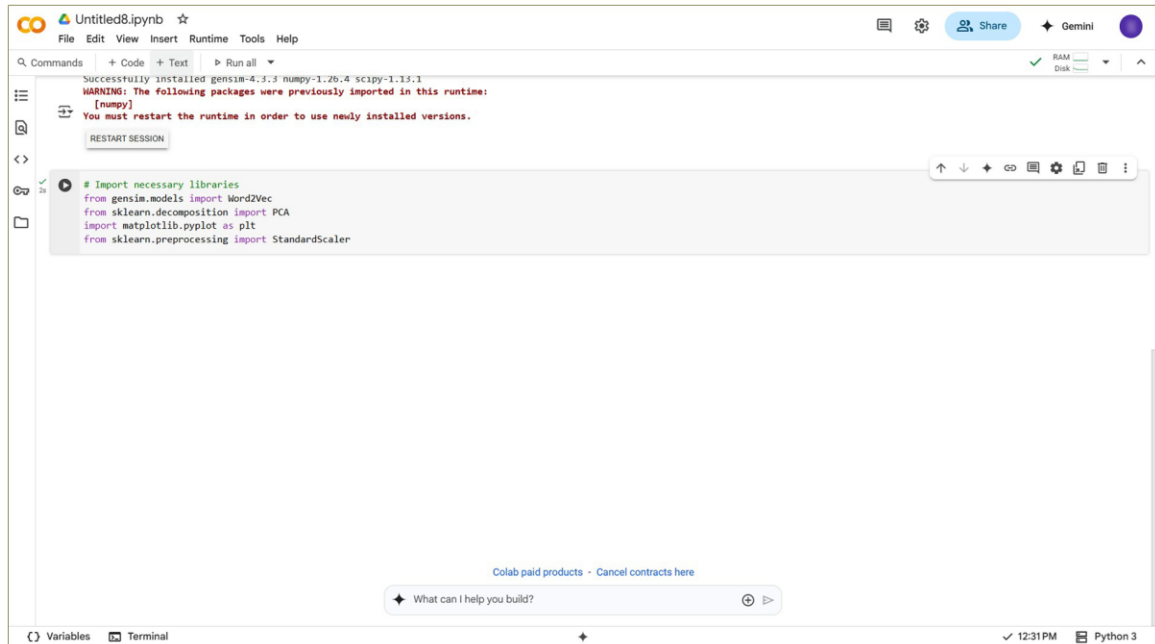
```
# Import necessary libraries
from gensim.models import Word2Vec
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
```

The following screenshot shows the code pasted into the code cell.



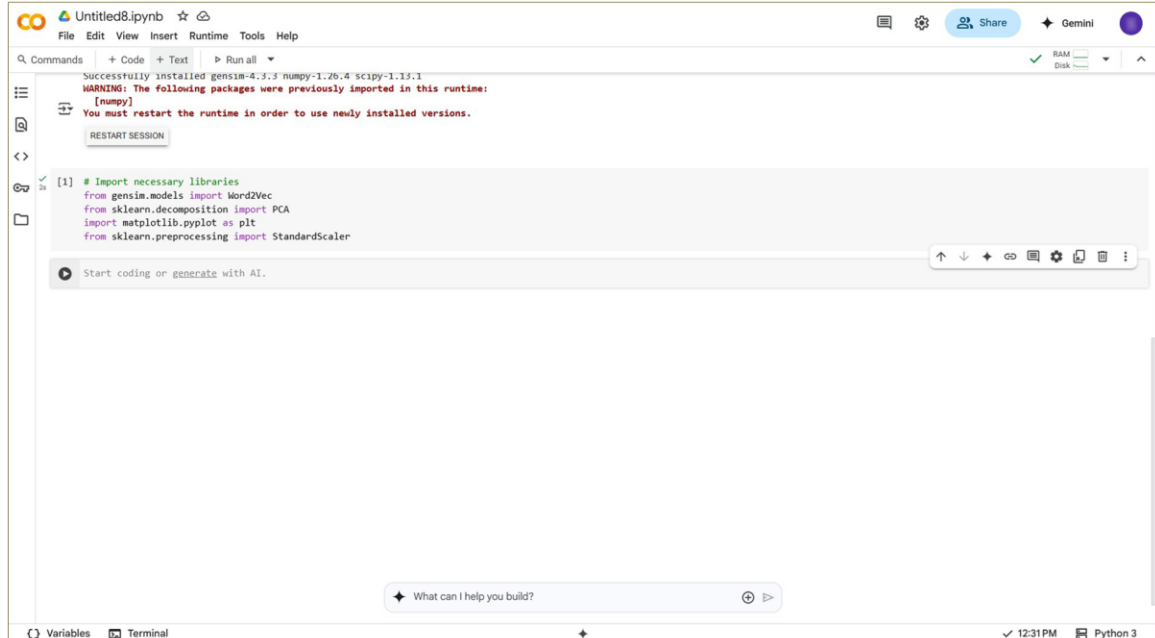
Result: The Word2Vec model is successfully imported. There isn't an output for running this code because it simply loads Word2Vec and PCA into the notebook's memory. A green checkmark is displayed next to the **Play** icon, which indicates that the code has been executed successfully.

Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**



5. Define the corpus, or collection of texts, for this embedding model. To do so, add a new cell in the notebook. Select the **+ Code** tab.

The following screenshot shows a new cell in the notebook.

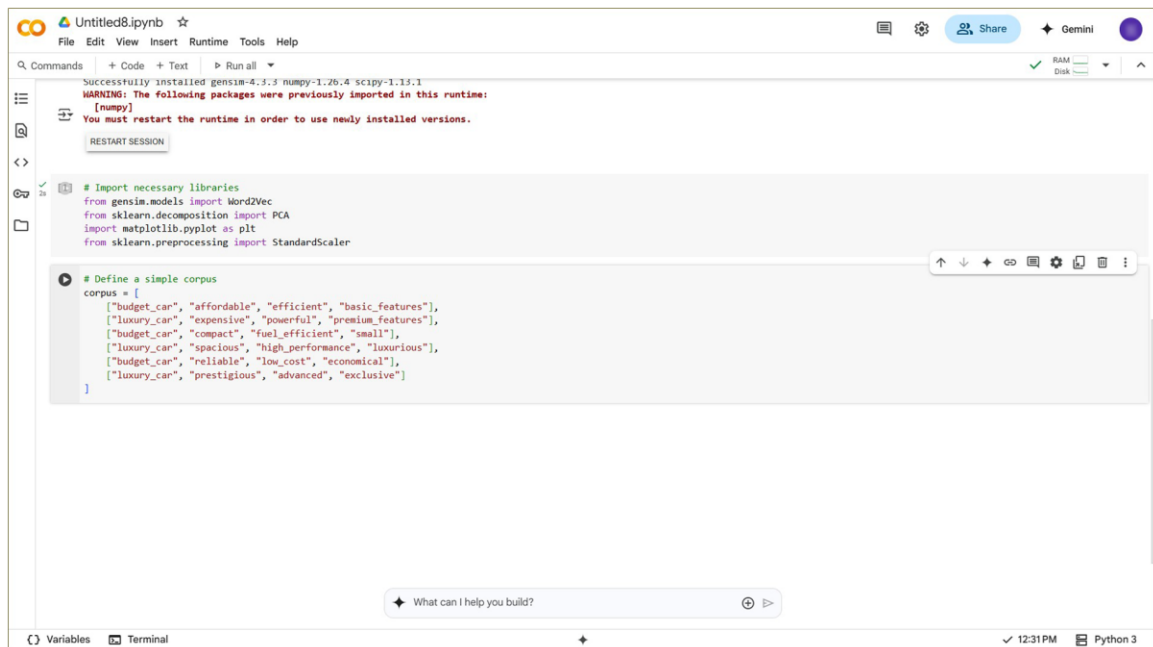


Copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
# Define a simple corpus
corpus = [
```

```
[ "budget_car", "affordable", "efficient", "basic_features"],  
[ "luxury_car", "expensive", "powerful", "premium_features"],  
[ "budget_car", "compact", "fuel_efficient", "small"],  
[ "luxury_car", "spacious", "high_performance", "luxurious"],  
[ "budget_car", "reliable", "low_cost", "economical"],  
[ "luxury_car", "prestigious", "advanced", "exclusive"]  
]
```

The following screenshot shows the code pasted into the code cell.



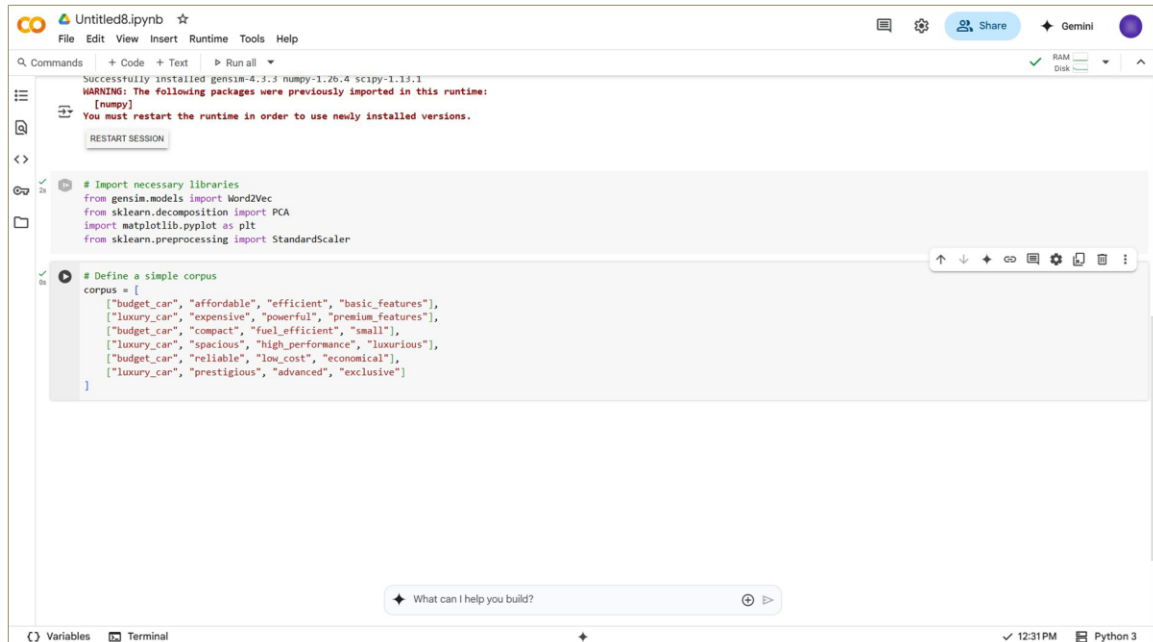
The screenshot shows a Google Colab notebook interface. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar, there's a status bar indicating 'Successfully installed gensim-4.3.3 numpy-1.26.4 scipy-1.13.1' and a warning: 'WARNING: The following packages were previously imported in this runtime: [numpy] You must restart the runtime in order to use newly installed versions.' A 'RESTART SESSION' button is visible. The main code cell contains the following Python code:

```
# Import necessary libraries  
from gensim.models import Word2Vec  
from sklearn.decomposition import PCA  
import matplotlib.pyplot as plt  
from sklearn.preprocessing import StandardScaler  
  
# Define a simple corpus  
corpus = [  
    ["budget_car", "affordable", "efficient", "basic_features"],  
    ["luxury_car", "expensive", "powerful", "premium_features"],  
    ["budget_car", "compact", "fuel_efficient", "small"],  
    ["luxury_car", "spacious", "high_performance", "luxurious"],  
    ["budget_car", "reliable", "low_cost", "economical"],  
    ["luxury_car", "prestigious", "advanced", "exclusive"]  
]
```

At the bottom of the notebook, there's a search bar with the text 'What can I help you build?' and a 'Python 3' indicator.

Result: The corpus is defined. There isn't an output for running this code because it simply loads the corpus into the notebook's memory.

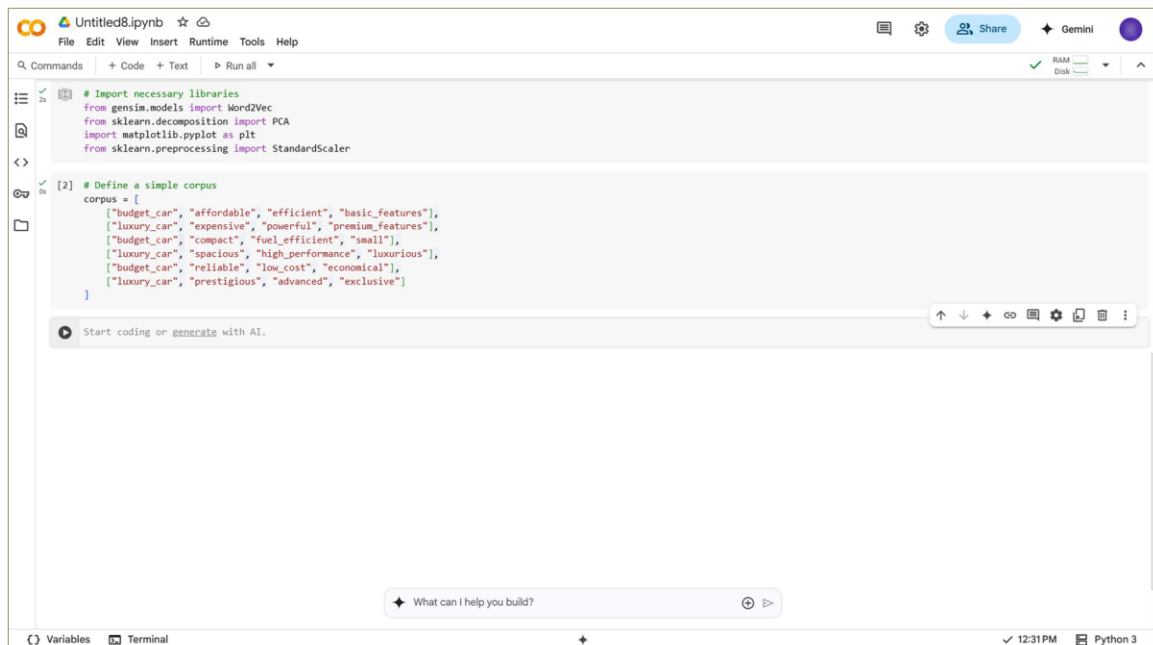
Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**



Alt text: The notebook displays the code in the code cell.

6. Train the Word2Vec model using the default parameters. To do so, add a new cell in the notebook. Select the **+ Code** tab.

The following screenshot shows a new cell in the notebook.

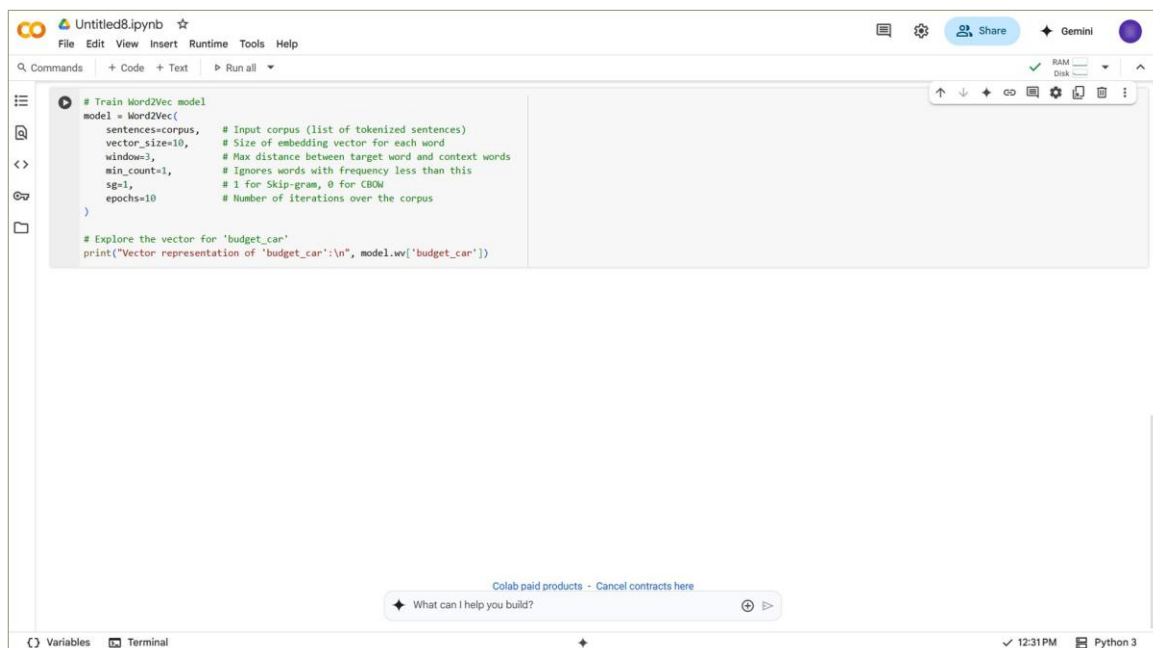


Copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
# Train Word2Vec model
model = Word2Vec(
```

```
sentences=corpus, # Input corpus (list of tokenized
sentences)
vector_size=10, # Size of embedding vector for each word
window=3, # Max distance between target word and context
words
min_count=1, # Ignores words with frequency less than this
sg=1, # 1 for Skip-gram, 0 for CBOW
epochs=10 # Number of iterations over the corpus
)
# Explore the vector for 'budget_car'
print("Vector representation of 'budget_car':\n",
model.wv['budget_car'])
```

The following screenshot shows the code pasted into the code cell.



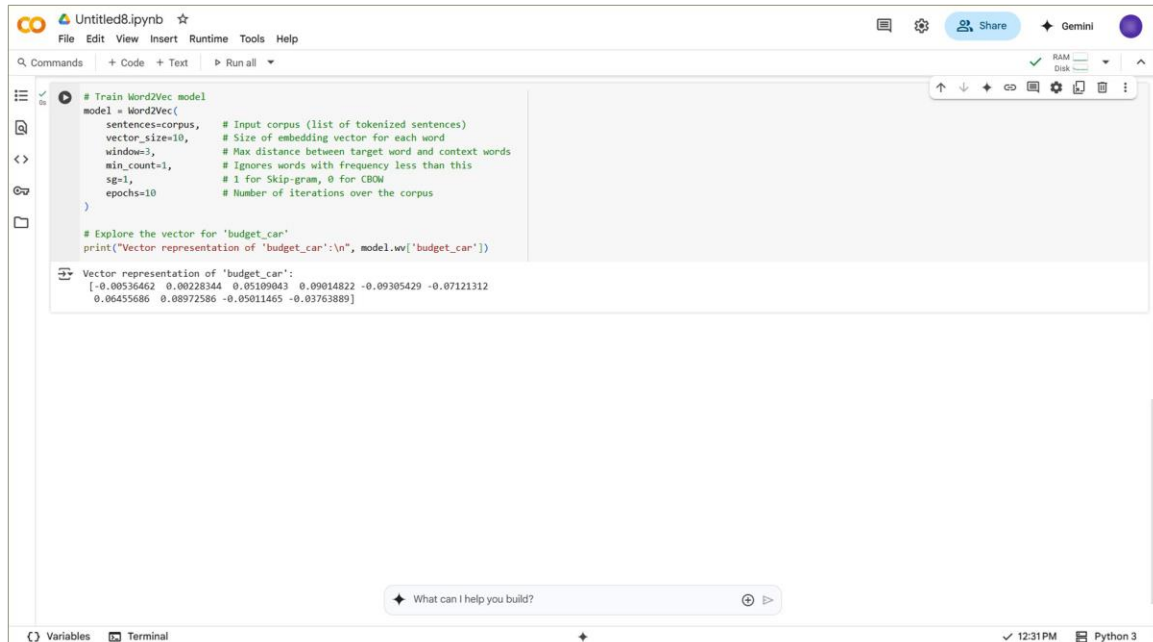
The screenshot displays a Google Colab notebook interface. The top bar includes the Colab logo, the file name 'Untitled8.ipynb', and standard menu options like File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu is a toolbar with icons for commands, code, text, and running. The main code cell contains the following Python code:

```
# Train Word2Vec model
model = Word2Vec(
    sentences=corpus, # Input corpus (list of tokenized sentences)
    vector_size=10, # Size of embedding vector for each word
    window=3, # Max distance between target word and context words
    min_count=1, # Ignores words with frequency less than this
    sg=1, # 1 for Skip-gram, 0 for CBOW
    epochs=10 # Number of iterations over the corpus
)

# Explore the vector for 'budget_car'
print("Vector representation of 'budget_car':\n", model.wv['budget_car'])
```

The bottom of the notebook shows a terminal area with the prompt 'What can I help you build?' and a status bar at the very bottom indicating 'Variables', 'Terminal', '12:31 PM', and 'Python 3'.

Result: Word2Vec is trained using the default parameters. Review the output that the model generated. As shown in the screenshot, the default parameters provide only the numerical representation of a word.



The screenshot shows a Google Colab notebook titled 'Untitled8.ipynb'. The code in the notebook is as follows:

```
# Train Word2Vec model
model = Word2Vec(
    sentences=corpus,      # Input corpus (list of tokenized sentences)
    vector_size=10,       # Size of embedding vector for each word
    window=3,             # Max distance between target word and context words
    min_count=1,           # Ignores words with frequency less than this
    sg=1,                  # 1 for Skip-gram, 0 for CBOW
    epochs=10              # Number of iterations over the corpus
)

# Explore the vector for 'budget_car'
print("Vector representation of 'budget_car':\n", model.wv['budget_car'])
```

The output of the notebook shows the vector representation of 'budget_car':

```
Vector representation of 'budget_car':
[[-0.00536462  0.00228344  0.05109043  0.09014822 -0.09305429 -0.07121312
  0.06455686  0.08972586 -0.05011465 -0.03763889]]
```

Visualize word embeddings using dimensionality reduction techniques

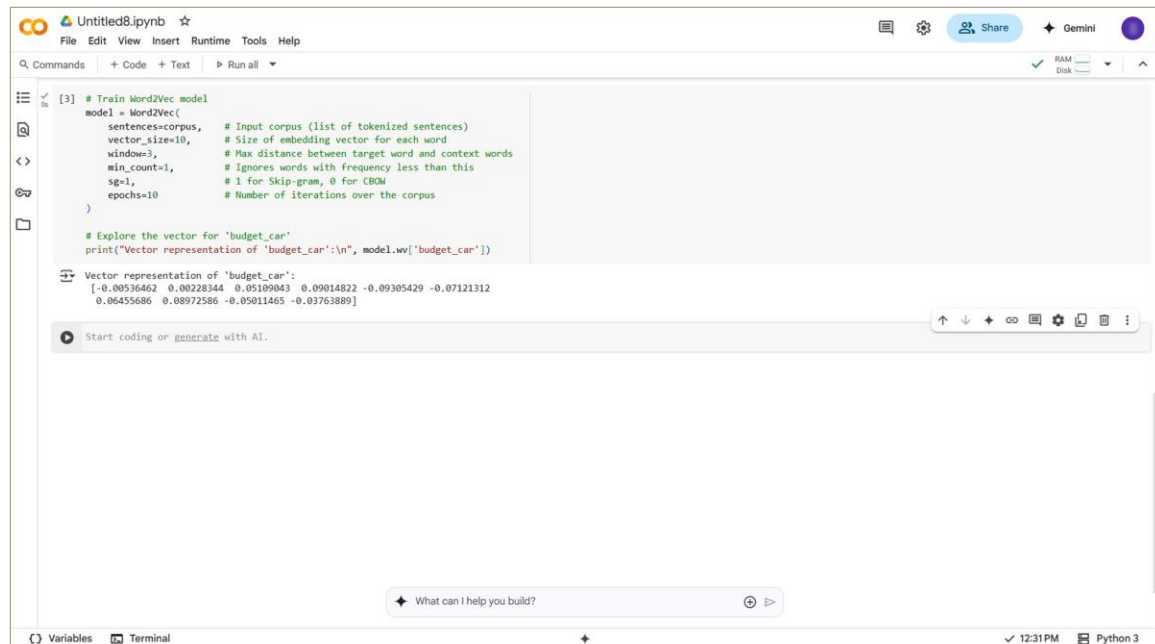
Overview

In this step, you'll use dimensionality reduction techniques such as PCA and t-SNE to observe word clusters and relationships. You'll first use PCA, followed by t-SNE.

Instructions

1. Apply the PCA technique to reduce dimensions. To do so, add a new cell in the notebook. Select the **+ Code** tab.

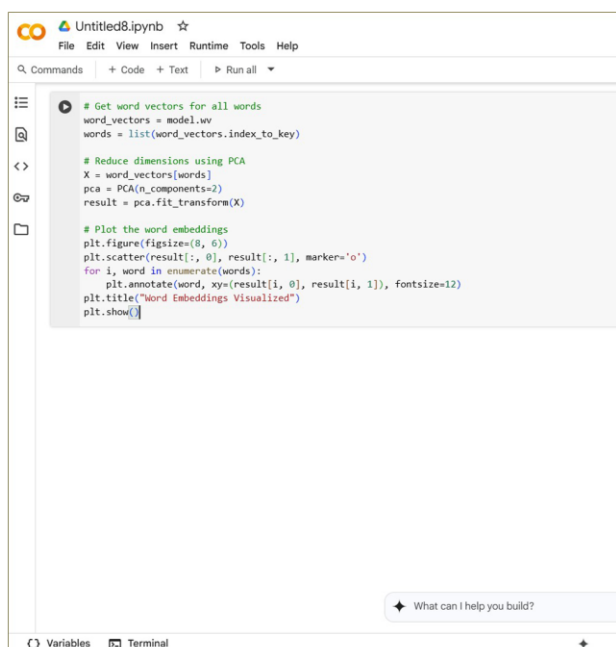
The following screenshot shows a new cell in the notebook.



Copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
# Get word vectors for all words
word_vectors = model.wv
words = list(word_vectors.index_to_key)
# Reduce dimensions using PCA
X = word_vectors[words]
pca = PCA(n_components=2)
result = pca.fit_transform(X)
# Plot the word embeddings
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], marker='o')
for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]),
        fontsize=12)
plt.title("Word Embeddings Visualized")
plt.show()
```

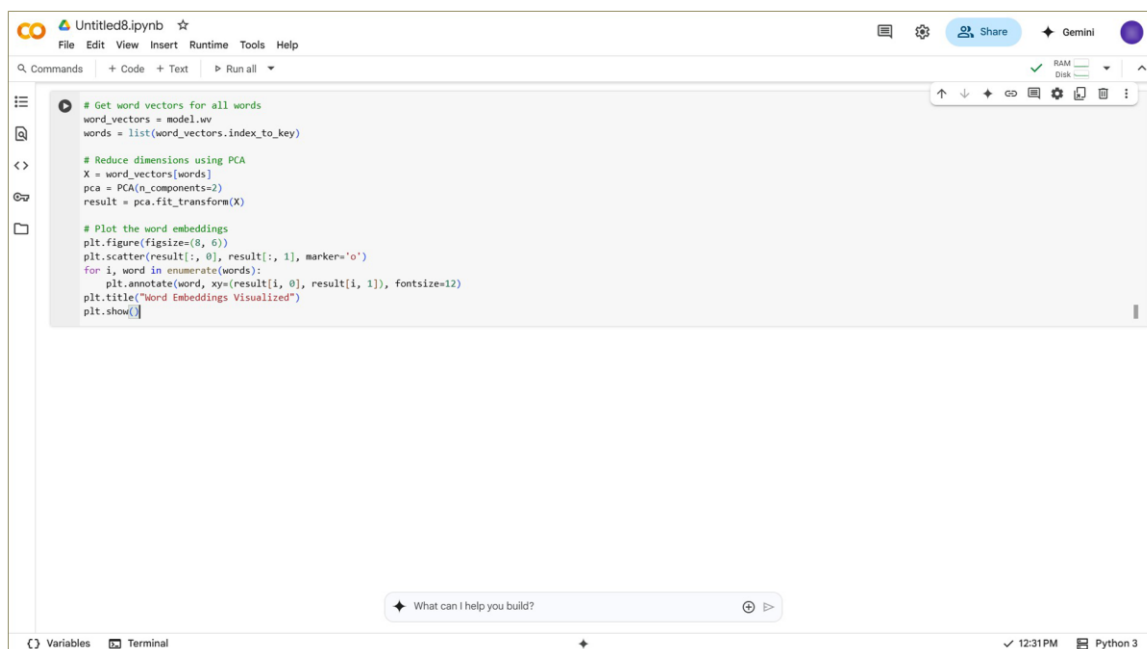
The following screenshot shows the code pasted into the code cell.



```
# Get word vectors for all words
word_vectors = model.wv
words = list(word_vectors.index_to_key)

# Reduce dimensions using PCA
X = word_vectors[words]
pca = PCA(n_components=2)
result = pca.fit_transform(X)

# Plot the word embeddings
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], marker='o')
for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]), fontsize=12)
plt.title("Word Embeddings Visualized")
plt.show()
```



```
# Get word vectors for all words
word_vectors = model.wv
words = list(word_vectors.index_to_key)

# Reduce dimensions using PCA
X = word_vectors[words]
pca = PCA(n_components=2)
result = pca.fit_transform(X)

# Plot the word embeddings
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], marker='o')
for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]), fontsize=12)
plt.title("Word Embeddings Visualized")
plt.show()
```

Result: The model generates the output that confirms the successful use of the PCA technique to reduce dimensionality. As shown in the following screenshot, the output doesn't show distinct word clusters for budget-friendly and luxury cars.

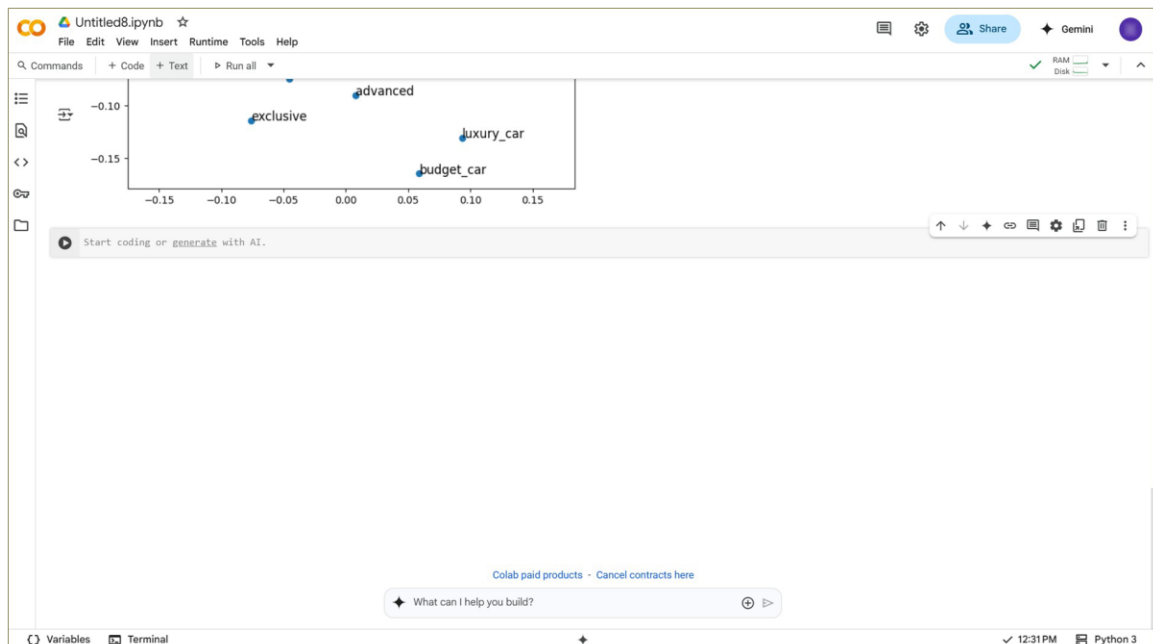
Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**



2. Apply the t-SNE technique to get well-defined clusters.

To add a new cell in the notebook, select the **+ Code** tab.

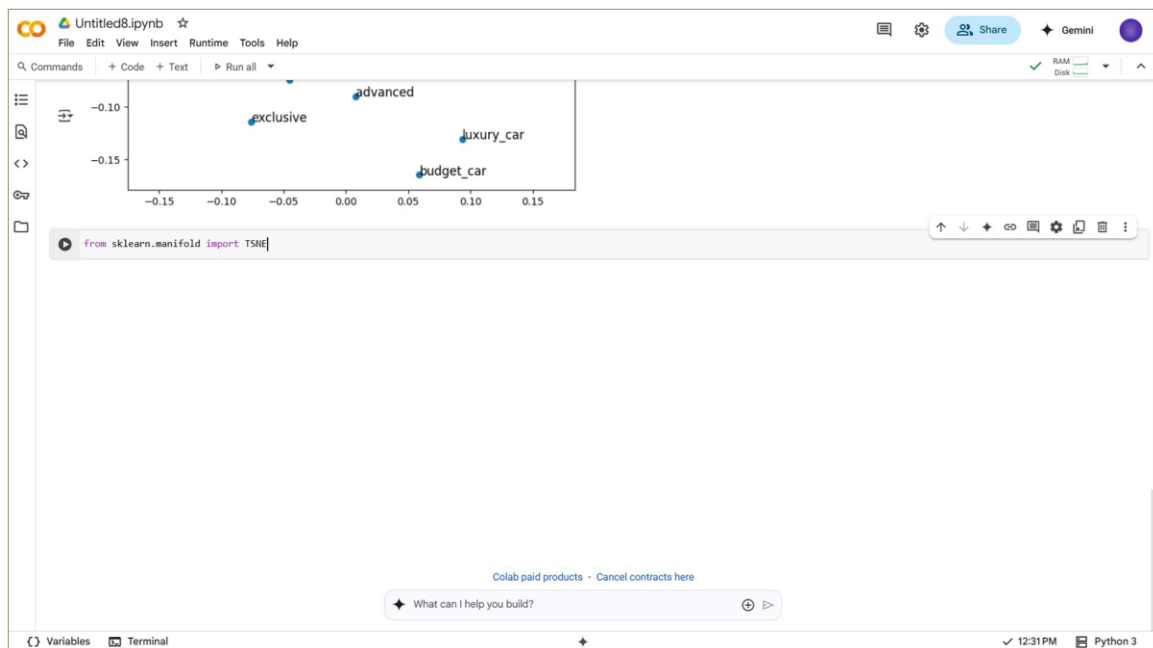
The following screenshot shows a new cell in the notebook.



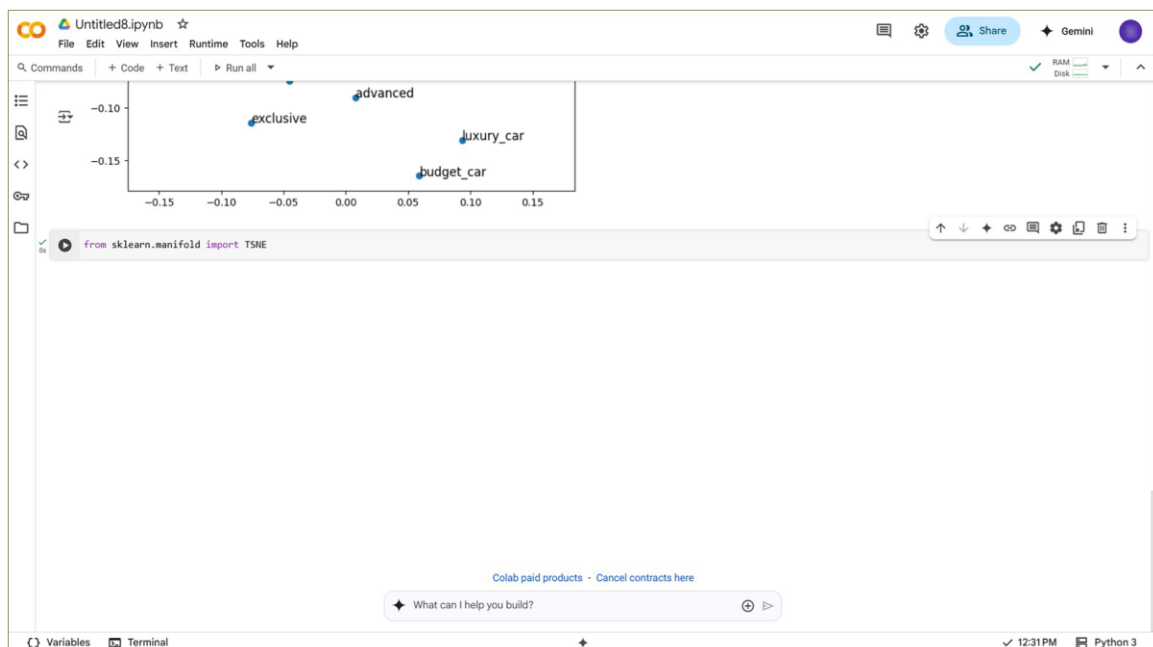
Copy the following code, paste it into the code cell, and then select the **Play** icon to execute it.

```
from sklearn.manifold import TSNE
```


The following screenshot shows the code pasted into the code cell.

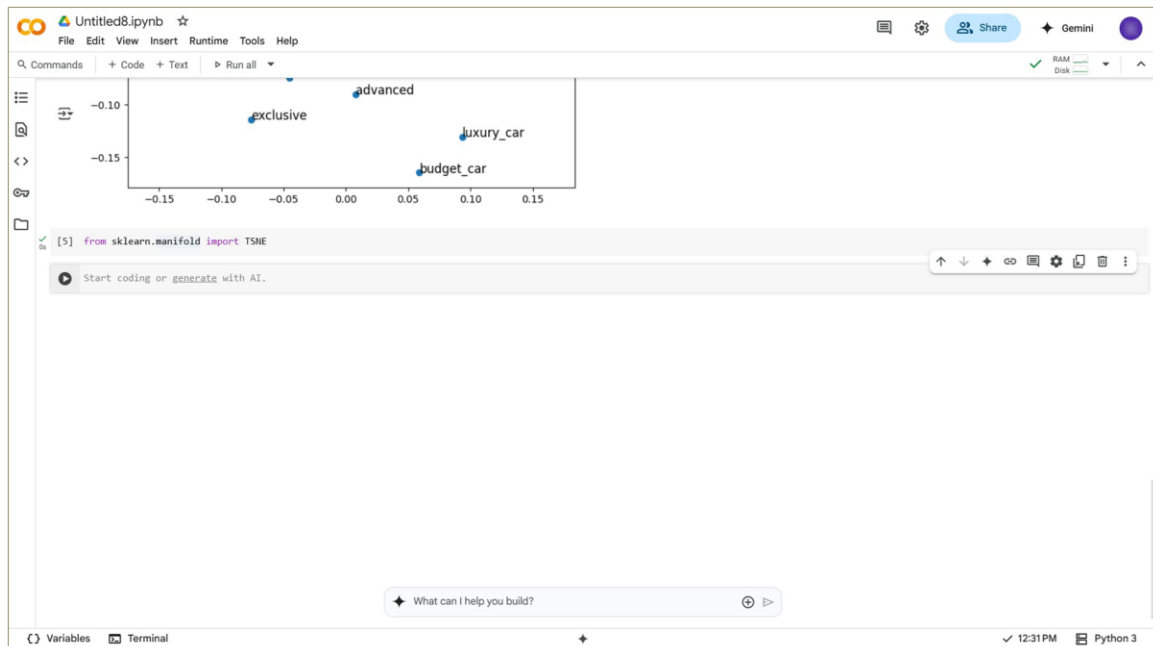


Result: There isn't an output for running this code because it simply loads t-SNE into the notebook's memory. As shown in the screenshot below, a green checkmark next to the **Play** icon indicates that the code has been executed successfully.



3. Change some parameters, such as increasing the vector size and using Continuous Bag of Words (CBOW). To add a new cell in the notebook. Select the **+ Code** tab.

The following screenshot shows a new cell in the notebook.

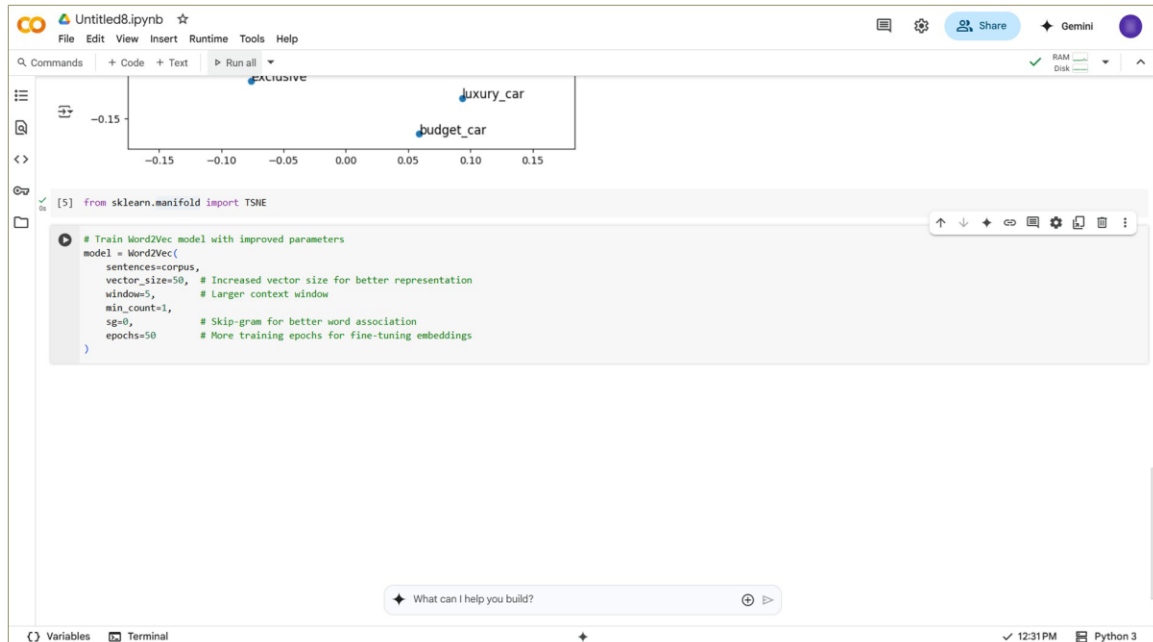


To do so, copy the following code into the new code cell and select the **Play** icon to execute it.

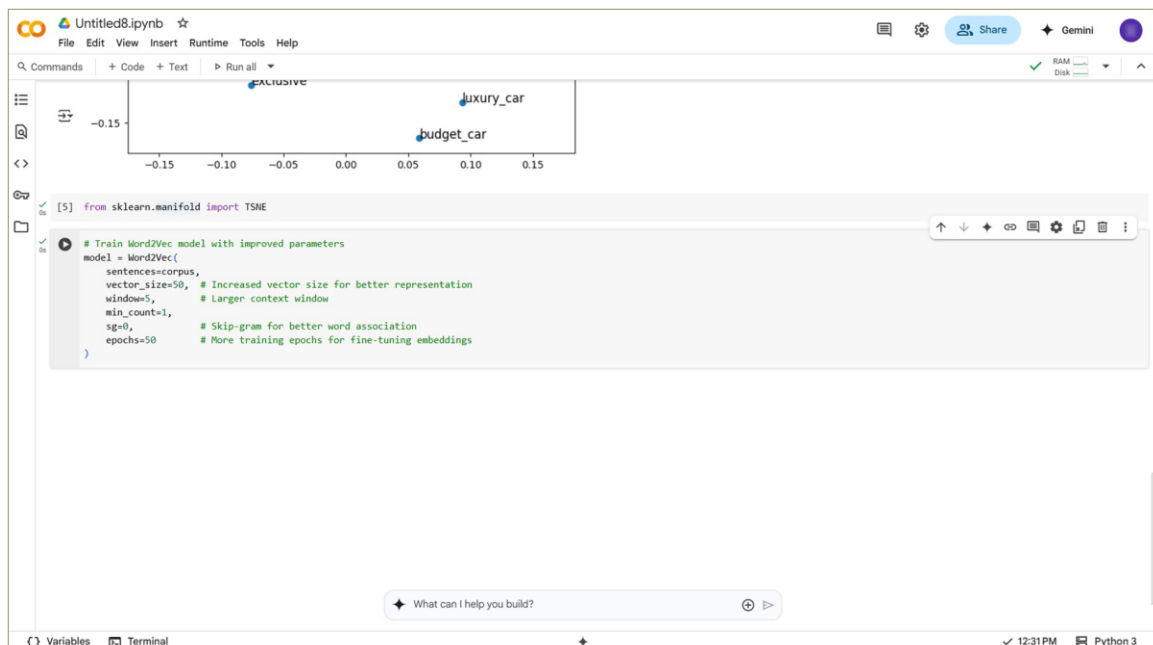
```
# Train Word2Vec model with improved parameters
model = Word2Vec(
    sentences=corpus,
    vector_size=50, # Increased vector size for better
representation
    window=5,      # Larger context window
    min_count=1,
    sg=0,          # Skip-gram for better word association
    epochs=50      # More training epochs for fine-tuning embeddings
)
```

The following screenshot shows the code pasted into the code cell.

Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**

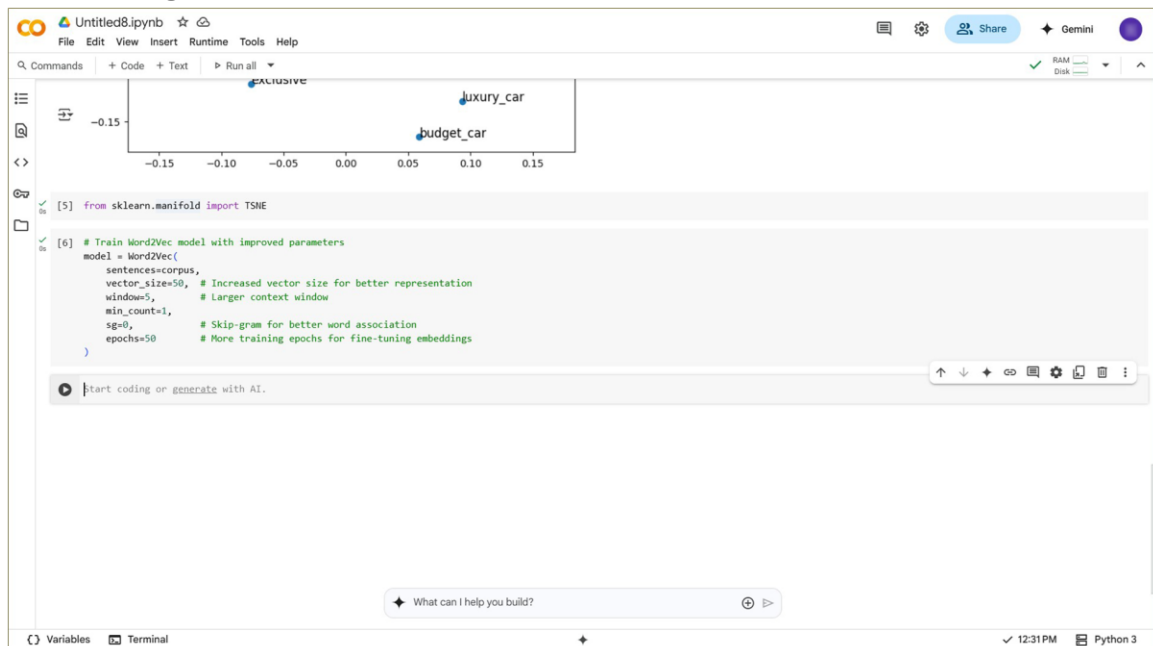


Result: The parameters have been changed. There isn't an output for running this code because it simply loads the parameters into the notebook's memory. As shown in the screenshot below, a green checkmark next to the **Play** icon indicates that the code has been executed successfully.



4. Apply t-SNE to fine-tune the visualization. To do so, add a new cell in the notebook. Select the + Code tab.

The following screenshot shows a new cell in the notebook.



Copy the following code into the new code cell and select the **Play** icon to execute it.

```
# Get word vectors
word_vectors = model.wv
words = list(word_vectors.index_to_key)
X = word_vectors[words]
# Reduce dimensions using t-SNE
tsne = TSNE(n_components=2, perplexity=3, random_state=42)
result = tsne.fit_transform(X)
# Plot the word embeddings
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], marker='o')
for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]),
        fontsize=12)
plt.title("Word Embeddings Visualized with t-SNE")
plt.show()
```

The following screenshot shows the code pasted into the code cell.

Implement a text embedding-based search system using Google Colab **IBM SkillsBuild**

```
from sklearn.manifold import TSNE

# Train Word2Vec model with improved parameters
model = Word2Vec(
    sentences=corpus,
    vector_size=50, # Increased vector size for better representation
    window=5, # Larger context window
    min_count=1,
    sg=0, # Skip-gram for better word association
    epochs=50 # More training epochs for fine-tuning embeddings
)

# Get word vectors
word_vectors = model.wv
words = list(word_vectors.index_to_key)
X = word_vectors[words]

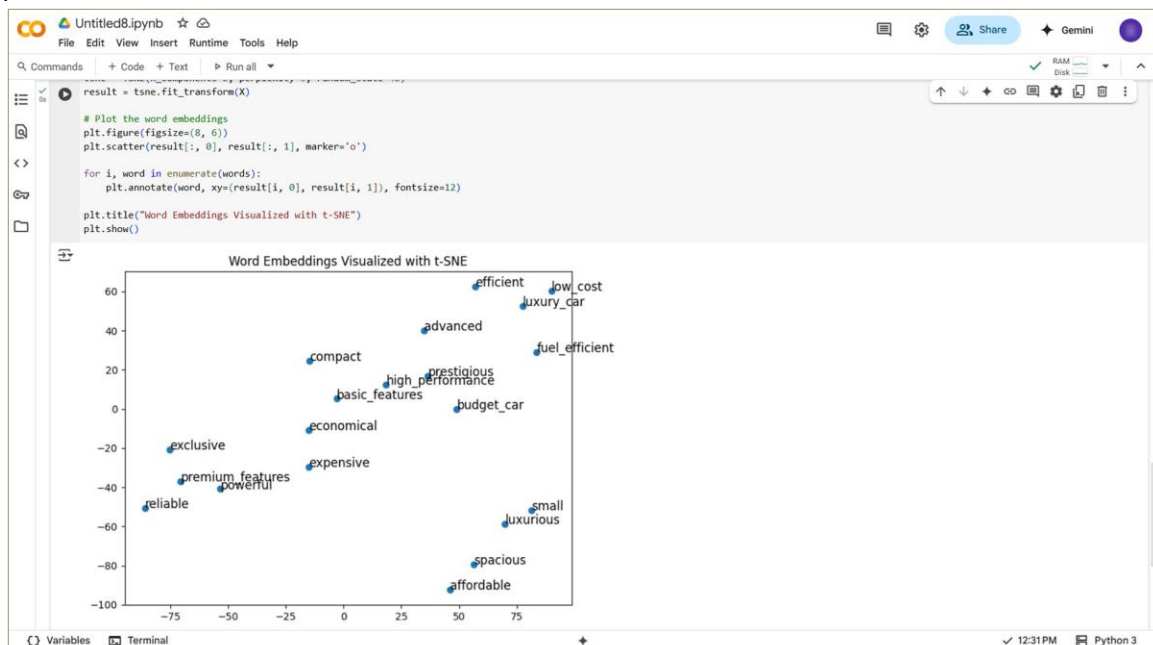
# Reduce dimensions using t-SNE
tsne = TSNE(n_components=2, perplexity=3, random_state=42)
result = tsne.fit_transform(X)

# Plot the word embeddings
plt.figure(figsize=(8, 6))
plt.scatter(result[:, 0], result[:, 1], marker='o')

for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]), fontsize=12)

plt.title("Word Embeddings Visualized with t-SNE")
plt.show()
```

Result: t-SNE to fine-tuned the visualization. Review the output that the model generated. As shown in the following screenshot, using t-SNE and refining the parameters results in more well-defined clusters.



Conclusion

Congratulations! You've successfully implemented a text embedding-based search system to retrieve relevant results using Google Colab. You started by training Word2Vec, a text embedding model, on a sample corpus containing the words customers use to describe budget-friendly and luxury cars. You used default parameters to generate word embeddings. Then, you visualized these embeddings using dimensionality reduction techniques, such as PCA and t-SNE, and refined the model parameters. This revealed two distinct clusters of words that frequently appear together. You also explored ways to fine-tune the model to make these clusters more distinct and uncover even deeper patterns in customer preferences.